BRIEFING START

Build a votes data pipeline — from scratch.

A two-part data engineering exercise. No frameworks. No shortcuts. Show us how you think about data at scale.

~2-3h	2	DuckDB
expected time	independent parts	the only DB you need

Part 1 · Ingestion

Build an incremental load process for a JSONL vote file. Handle inserts, handle updates. No duplicates downstream — ever.

Part 2 · Outlier detection

Write a SQL view that identifies weeks where vote counts deviate from the weekly mean by more than 20%.

What we're really testing

Python structure, SQL fluency, test quality, and how you communicate your decisions. Not whether you can Google a DuckDB upsert.

SECTION 01

The two parts

Each part must be runnable independently. Ingestion shouldn't trigger outlier calculation. Outlier calc assumes data is already ingested.

PART 1 · INGESTION

Load votes into DuckDB

Build an on-demand ingestion process that reads vote data from a JSONL file and loads it into the `blog_analysis.votes` table. It must handle new records and updates to existing records — safely, repeatedly, without duplicates.

PART 2 · OUTLIERS

Identify anomalous weeks in SQL

Create a SQL view `blog_analysis.outlier_weeks` that surfaces only the weeks where vote totals deviate significantly from the dataset average. Running `outliers.py` should recreate the view and print its contents — nothing more.

SECTION 02

Hard constraints

✓ DO THIS

- → Pure Python + DuckDB + SQL
- → Upsert logic — inserts AND updates
- → Self-contained, order-independent tests
- → Schema: `blog_analysis`
- → OLAP-style datatypes and loading
- → Explain non-obvious choices in README
- → Disclose AI tool usage honestly

✗ DON'T DO THIS

- ✗ pandas, polars, PySpark, Ballista
- ✗ Unnecessary abstraction layers
- ✗ Custom test frameworks
- ✗ Over-engineering — it's a toy problem
- ✗ Editing `tests/exercise-tests/`
- ✗ Submitting code you can't explain

⚡ ON AI USAGE

AI is fine to use — for pairing, debugging, exploring queries. But in the follow-up interview you'll walk through every line live, explain your decisions, and make changes on the spot. Use it, own it.

SECTION 03

How we evaluate

We start with automated tests, then read the code. These are the lenses we use — in order of weight.

Test quality — not just coverage

Tests that verify actual behaviour. Self-contained. No shared state, no ordering dependency. A test that only "runs" without asserting anything meaningful fails this bar.

SQL fluency

We're pretending DuckDB is an OLAP warehouse. Pick datatypes, data models, and loading strategies you'd use in a real analytics system — not the first thing that works.

Simplicity as a virtue

YAGNI. KISS. DRY. Solutions should reflect the difficulty of the problem. If you've added an abstraction nobody asked for, it counts against you. The best code here will be boring.

Self-explanatory code

If your README needs three paragraphs to explain what your code does, the code isn't clear enough. Explain your *choices* (why this data model? why this loading strategy?) — not the mechanics.

Handling ambiguity

If something in the spec is unclear, make a decision, document your assumption, and move forward. Asking before you've tried to figure it out yourself isn't the move.

SECTION 04

Before you submit

Work through this before pushing. Every item is either a hard requirement or something that'll come up in the review interview.

Submission checklist

reset

- [ ] **Both parts run independently** — required
  Ingestion doesn't trigger outlier calc. Each can be run standalone without errors.
- [ ] **No pandas / polars / dataframe libraries** — required
  Grep your imports if you're not sure. One import will disqualify the submission.
- [ ] **exercise check-ingestion passes** — required
  Run `poetry run exercise check-ingestion` — should pass clean.
- [ ] **exercise check-outliers passes** — required
  Run `poetry run exercise check-outliers` — verify against the sample dataset first.
- [ ] **Tests are self-contained and order-independent** — required
  Run your tests in reverse order. Run them individually. They should all pass regardless of sequence.
- [ ] **README covers approach, assumptions, AI usage** — expected
  Explain your data model choice, upsert strategy, and anything ambiguous you resolved. List AI tools used and how.